

Sistema de Gestión de Expedientes
Documentación Técnica — Parcial 2

Curso	Bases de Datos
Entrega	Parcial 2 — Clase v0.2
Año	2024
Tecnologías	MySQL · Python · Flask · REST API

TABLA DE CONTENIDOS

1. Diseño de la Base de Datos
2. Diagrama Entidad-Relación (ER)
3. Normalización — 1FN, 2FN, 3FN
4. Integración Python con MySQL
5. Vistas SQL (5 Vistas)
6. API REST con Flask — PMV
7. 5 Lecciones Aprendidas
8. Conclusiones

1. DISEÑO DE LA BASE DE DATOS

El sistema de gestión de expedientes de Delgado & Delgado requiere una base de datos relacional que soporte los módulos de: Login, Expedientes, Aseguradoras, Tipos de Caso y Abogados (Licenciados). Se diseñaron **11 tablas** siguiendo buenas prácticas de normalización hasta la Tercera Forma Normal (3FN).

Tablas del Sistema

Tabla	Descripción	Registros Clave
roles	Catálogo de roles del sistema	id_rol, nombre_rol
usuarios	Usuarios con credenciales de acceso	id_usuario, username, password_hash, id_rol
aseguradoras	Compañías aseguradoras	id_aseguradora, nombre, codigo
ciudades	Catálogo de ciudades	id_ciudad, nombre
juzgados	Juzgados por ciudad	id_juzgado, nombre, numero, id_ciudad
tipos_caso	Tipos de proceso (Tránsito/Penal)	id_tipo_caso, nombre
abogados	Licenciados vinculados a usuarios	id_abogado, id_usuario, titulo
conductores	Personas involucradas en casos	id_conductor, nombre, cedula
estados_expediente	Estados posibles de un expediente	id_estado, nombre, color_hex
expedientes	Entidad central del sistema	id_expediente, codigo_expediente, ...
agenda	Citas y audiencias del día	id_agenda, id_expediente, fecha, hora
documentos_expediente	Archivos adjuntos	id_documento, id_expediente, ruta

2. DIAGRAMA ENTIDAD-RELACIÓN

El siguiente diagrama muestra las relaciones entre las entidades del sistema. La entidad central es **EXPEDIENTES**, que se relaciona con todas las demás entidades mediante claves foráneas.

ENTIDAD	RELACIÓN	ENTIDAD	CARDINALIDAD
usuarios	■■■tiene■■■	roles	N : 1
abogados	■■■es■■■	usuarios	1 : 1
expedientes	■■■asignado a■■■	abogados	N : 1
expedientes	■■■tiene■■■	conductores	N : 1
expedientes	■■■cubre■■■	aseguradoras	N : 1
expedientes	■■■es de tipo■■■	tipos_caso	N : 1
expedientes	■■■se lleva en■■■	juzgados	N : 1
expedientes	■■■tiene■■■	estados_expediente	N : 1
juzgados	■■■ubicado en■■■	ciudades	N : 1
agenda	■■■pertenece a■■■	expedientes	N : 1
documentos_expediente	■■■adjunto a■■■	expedientes	N : 1

Nota: El diagrama completo en formato gráfico (ERD) se incluye como anexo visual generado por MySQL Workbench. Las relaciones implementadas garantizan integridad referencial mediante FOREIGN KEY constraints en todas las tablas.

3. NORMALIZACIÓN — 1FN, 2FN, 3FN

Primera Forma Normal (1FN)

Una tabla está en 1FN cuando todos sus atributos son atómicos (no divisibles), no existen grupos repetitivos y tiene una clave primaria definida.

Aplicación en el sistema:

- Cada campo contiene un único valor atómico (ej: nombre y apellido son columnas separadas en conductores y usuarios).
- No se almacenan listas en un campo (ej: múltiples aseguradoras → tabla aseguradoras normalizada).
- Cada tabla tiene su PRIMARY KEY (id_expediente, id_abogado, etc.).
- Los juzgados y sus ciudades están en campos separados, no concatenados.

Segunda Forma Normal (2FN)

Una tabla está en 2FN cuando cumple 1FN y todos los atributos no clave dependen completamente de la clave primaria (no existen dependencias parciales).

- La tabla expedientes usa id_expediente como PK simple → todos los atributos dependen completamente de ella.
- No se usan claves compuestas innecesarias que causen dependencias parciales.
- Los datos del abogado (nombre, título) no están en expedientes; se referencian vía id_abogado → tabla abogados.
- Los datos de aseguradora no se repiten en cada expediente; se usa id_aseguradora como FK.

Tercera Forma Normal (3FN)

Una tabla está en 3FN cuando cumple 2FN y no existen dependencias transitivas (ningún atributo no clave depende de otro atributo no clave).

- La ciudad del juzgado se extrajo a tabla ciudades → juzgados.id_ciudad evita la transitividad ciudad→juzgado→expediente.
- El nombre del rol no está en usuarios directamente; se usa id_rol → tabla roles.
- La especialidad y título del abogado no están en usuarios; están en la tabla abogados vinculada por id_usuario.
- El color_hex del estado está en estados_expediente, no repetido en cada expediente.
- Todos los catálogos (tipos_caso, aseguradoras, juzgados) eliminan la redundancia de datos en expedientes.

Ejemplo Práctico de Normalización

Tabla sin normalizar (forma plana):

expediente_id	conductor	aseguradora	ciudad_juzgado	nombre_juzgado	abogado
EXP-001	Anthony Trejo	ASSA	Panamá	Juzgado 5to Pedregal	Dr. Diane Campbell
EXP-002	Angel Delgado	ANCON	Panamá	Juzgado 4to Pedregal	Dr. Harold Gray

Problema: Redundancia de datos, inconsistencia si cambia el nombre del juzgado, no hay integridad referencial. →

Solución: Tablas separadas con FK.

4. INTEGRACIÓN PYTHON CON MYSQL

La integración entre Python y MySQL se realiza mediante la librería **Flask-MySQLdb**, que provee un conector eficiente con soporte para cursores de diccionario (DictCursor), permitiendo resultados como objetos JSON.

Configuración de la Conexión

```
app.config['MYSQL_HOST'] = os.environ.get('DB_HOST', 'localhost')
app.config['MYSQL_USER'] = os.environ.get('DB_USER', 'root')
app.config['MYSQL_PASSWORD'] = os.environ.get('DB_PASS', '')
app.config['MYSQL_DB'] = os.environ.get('DB_NAME', 'delgado_abogados')
app.config['MYSQL_CURSORCLASS'] = 'DictCursor'
```

Patrón de Consulta

```
cur = mysql.connection.cursor()
cur.execute("SELECT * FROM vista_expedientes_completos WHERE id_expediente = %s", (id,))
result = cur.fetchone()
cur.close()
return jsonify({"data": result})
```

Librerías Utilizadas

Librería	Versión	Propósito
Flask	3.0.0	Framework web para la API REST
Flask-MySQLdb	2.0.0	Conector MySQL para Flask
PyJWT	2.8.0	Generación y validación de tokens JWT
bcrypt	4.1.2	Hash seguro de contraseñas
python-dotenv	1.0.0	Manejo de variables de entorno

5. VISTAS SQL

Se implementaron **5 vistas SQL** que simplifican las consultas de la API y encapsulan la lógica de JOIN entre tablas relacionadas.

Vista 1: vista_expedientes_completos

Consolida todos los datos de un expediente: conductor, aseguradora, abogado, juzgado, ciudad y estado. Es la vista principal usada por el endpoint GET /api/expedientes.

```
SELECT e.*, c.nombre AS conductor, a.nombre AS aseguradora, ... FROM expedientes e JOIN
conductores c ON ... JOIN aseguradoras a ON ... [JOIN x5]
```

Vista 2: vista_agenda_del_dia

Muestra las citas programadas del día con información del expediente, conductor, abogado y juzgado. Usada en el dashboard principal.

```
SELECT ag.fecha, ag.hora, e.codigo_expediente, conductor, juzgado FROM agenda ag JOIN expedientes
e ON ... WHERE fecha = CURDATE()
```

Vista 3: vista_resumen_estados

Conteo de expedientes agrupados por estado (Pendiente, En Curso, Cerrado). Alimenta los contadores del dashboard: 102 / 72 / 204.

```
SELECT est.nombre, COUNT(e.id_expediente) AS total FROM estados_expediente LEFT JOIN expedientes
e ON ... GROUP BY est.id_estado
```

Vista 4: vista_expedientes_por_abogado

Carga de trabajo por abogado: total de expedientes, cuántos en curso, pendientes y cerrados. Usada en el módulo de Reportes.

```
SELECT abogado_completo, COUNT(*) total, SUM(CASE en_curso) ... FROM abogados JOIN usuarios ...
LEFT JOIN expedientes GROUP BY id_abogado
```

Vista 5: vista_expedientes_por_aseguradora

Expedientes totales por compañía aseguradora con desglose por estado. Usada en el módulo de Aseguradoras y Reportes.

```
SELECT a.nombre, COUNT(e.*), SUM(en_curso), SUM(pendientes) FROM aseguradoras a LEFT JOIN
expedientes e GROUP BY a.id_aseguradora
```

6. API REST CON FLASK — PRODUCTO MÍNIMO VIABLE

Se implementó una API REST con Flask que cubre todos los módulos del prototipo. La autenticación se realiza con **JWT (JSON Web Tokens)** y todas las rutas protegidas requieren el header **Authorization: Bearer <token>**.

Endpoints Implementados

Método	Endpoint	Descripción	Auth
POST	/api/auth/login	Autenticar usuario → retorna JWT	No
GET	/api/auth/me	Datos del usuario autenticado	Sí
GET	/api/expedientes	Listar expedientes (filtros + paginación)	Sí
GET	/api/expedientes/:id	Detalle de un expediente	Sí
POST	/api/expedientes	Crear nuevo expediente (3 pasos)	Sí
PUT	/api/expedientes/:id	Actualizar expediente	Sí
GET	/api/aseguradoras	Listar aseguradoras con estadísticas	Sí
POST	/api/aseguradoras	Crear nueva aseguradora	Sí
GET	/api/tipos-caso	Listar tipos de caso	Sí
POST	/api/tipos-caso	Crear tipo de caso	Sí
GET	/api/abogados	Listar abogados con carga de trabajo	Sí
POST	/api/abogados	Registrar nuevo abogado	Sí
GET	/api/conductores	Listar/buscar conductores	Sí
POST	/api/conductores	Crear nuevo conductor	Sí
GET	/api/juzgados	Listar juzgados activos	Sí
GET	/api/dashboard	Contadores + agenda del día	Sí
GET	/api/reportes/expedientes-por-aseguradora	Reporte aseguradoras	Sí
GET	/api/reportes/expedientes-por-abogado	Reporte abogados	Sí

Ejemplo: Crear Expediente (POST /api/expedientes)

Request Body (JSON):

```
{ "codigo_expediente": "EXP-999", "id_conductor": 1,
  "id_aseguradora": 1, "numero_caso": "CASO-2024-001",
  "id_tipo_caso": 1, "id_abogado": 2,
  "id_juzgado": 5, "fecha_inicio": "2024-01-15" }
```

Response (201 Created):

```
{ "success": true, "message": "Expediente creado exitosamente",
  "data": { "id_expediente": 11 } }
```

7. LECCIONES APRENDIDAS

Lección 1: La normalización ahorra problemas a largo plazo

Al inicio del diseño, la tentación es crear una tabla plana con todos los campos del expediente (aseguradora, juzgado, abogado concatenados). Sin embargo, esto genera redundancia masiva de datos y problemas de actualización (si cambia el nombre de una aseguradora, hay que actualizar cientos de filas). Aplicar 3FN desde el principio evita estas situaciones y hace el sistema más mantenible.

Lección 2: Las vistas SQL simplifican enormemente la capa de API

En lugar de escribir JOINS complejos en cada endpoint de Flask, crear vistas SQL reutilizables permite que los endpoints sean simples `SELECT * FROM vista_X`. Esto reduce el código Python, facilita el debugging y centraliza la lógica de consulta en un solo lugar (la base de datos).

Lección 3: JWT es superior a sesiones para APIs REST

Implementar autenticación con JWT (JSON Web Tokens) en lugar de sesiones de servidor permite que la API sea stateless (sin estado). Esto facilita el escalado horizontal, la integración con apps móviles y el consumo desde múltiples clientes. El token contiene el rol y datos del usuario, evitando consultas adicionales.

Lección 4: Las variables de entorno son esenciales para seguridad

Las credenciales de base de datos, llaves secretas de JWT y configuraciones sensibles NUNCA deben estar hardcodeadas en el código fuente. Usar `python-dotenv` y variables de entorno permite desplegar la misma aplicación en desarrollo, staging y producción sin modificar el código, y protege información sensible en repositorios de código.

Lección 5: El prototipo de UI es una guía invaluable para el diseño de datos

Analizar el prototipo del sistema (pantallas de Login, Dashboard, Nuevo Expediente) antes de diseñar la base de datos permitió identificar exactamente qué datos necesita cada pantalla. El formulario de 3 pasos del prototipo (Datos Generales, Anexos, Resumen) mapea directamente a los campos de la tabla expedientes y sus tablas relacionadas, haciendo el diseño más coherente con la experiencia de usuario.

8. CONCLUSIONES

El sistema de gestión de expedientes de Delgado & Delgado fue diseñado e implementado siguiendo las mejores prácticas de ingeniería de software y bases de datos relacionales. El proyecto demuestra la integración completa del stack tecnológico solicitado.

- Se diseñó una base de datos relacional con 11 tablas normalizadas hasta 3FN, eliminando redundancias y garantizando integridad referencial.
- Se implementaron 5 vistas SQL que encapsulan la lógica de negocio y simplifican el consumo desde la API.
- La API REST con Flask cubre todos los módulos del prototipo con 19 endpoints documentados, autenticación JWT y validación de datos.
- El uso de variables de entorno, hash de contraseñas con bcrypt y tokens JWT garantiza la seguridad del sistema.
- La arquitectura implementada es escalable y permite agregar nuevos módulos sin afectar los existentes.